Esterian Conquest

Sysop Manual

Version 1.0.0-beta.1 — Beta

Revision date: March 31, 2026

Contents

1. Introduction 2
2. Getting Started: Hosting a Campaign 2
   2.1. System Requirements 2
   2.2. Building from Source 2
   2.3. Current Beta Distribution 2
   2.4. Choose Your Deployment 3
      2.4.1. 1. Self-Host 3
      2.4.2. 2. Dedicated VPS Host 3
      2.4.3. 3. BBS Door Host 3
   2.5. Game Directory Layout 4
   2.6. Initializing a New Game 4
   2.7. Recommended Public Host 5
   2.8. Hosted Player Identity Management 6
3. Game Directory Structure 7
4. Configuration 7
      4.0.1. Full Example 7
      4.0.2. Stored Fields 7
      4.0.3. `session` Block 8
      4.0.4. `inactivity` Block 8
      4.0.5. `reservations` Block 8
   4.1. Display Defaults 9
5. SSH Access 9
6. Local and Direct Play 9
7. Legacy BBS Door Setup 9
   7.1. Flags for Door Mode 10
   7.2. Drop File 10
   7.3. Enigma BBS (Rust Client) 10
8. File-Based Turn Submission 11
9. Turn Processing and Maintenance 12
10. Player Management 12
   10.1. Reserving Seats 12
11. Terminology 13
12. CLI Reference 13
   12.1. ec-sysop 13
   12.2. ec-game 14

1. Introduction

Esterian Conquest (EC) is a multi-player strategy game originally written for DOS-era BBS systems. The Rust-native stack reimplements the game engine, player client, and admin tooling for modern systems.

This manual is for the **sysop** — the person responsible for hosting a game instance. It covers:

- choosing the right deployment path
- creating and maintaining a DB-only game
- running the recommended Nostr host
- running a local or direct SSH game
- putting `ec-game` on a BBS door
- managing players and yearly maintenance

For player-facing rules and gameplay, see the **Player Manual**.

This manual is the authoritative sysop manual for the Rust edition of Esterian Conquest. The preserved original `.DOC` set in `original/v1.5/` remains historical reference material and an ambiguity fallback for classic operator intent, not a higher-authority replacement for the current Rust manuals.

2. Getting Started: Hosting a Campaign

2.1. System Requirements

- Linux, macOS, or Windows
- Rust toolchain (stable) for normal self-host and VPS deployment
- For the recommended public-hosted flow: an SSH-accessible host plus a Nostr relay reachable by players
- A terminal emulator for localhost or direct SSH play
- A BBS server with socket support only if you specifically want legacy door deployment

2.2. Building from Source

From the repository root:

```
cargo build --release -p ec-sysop -p ec-game -p ec-connect
```

The release binaries will be in `target/release/`.

2.3. Current Beta Distribution

During the current beta, public GitHub Releases include Windows x64, Linux x64, and macOS Apple Silicon `ec-connect` player archives alongside the old DOS compatibility packages.

For the Rust edition:

1. Rust self-host and VPS sysops should build from tagged source with Cargo.
2. Hosted players can use the public GitHub Releases `ec-connect` archives with the player manual on Windows, Linux, or macOS.
3. BBS sysops should build from source or use a direct/private beta build.

A public Linux x64 BBS door package is planned later. When it ships, it should include `ec-game`, `ec-sysop`, the BBS wrapper/docs, the player manual, and this sysop manual.

2.4. Choose Your Deployment

EC has three practical deployment paths.

2.4.1. 1. Self-Host

Use this when a sysop wants to run one game for himself and a few friends.

1. Build the binaries.
2. Create a game directory with `ec-sysop new-game`.
3. Run `ec-game` directly, or turn on `ec-sysop nostr` if the players will connect remotely.
4. Run `ec-sysop maint` when the year should advance.

The simplest local setup is:

```
ec-sysop new-game /home/sysop/ec-games/friday-night --name "Friday Night EC" --players 4
ec-game --dir /home/sysop/ec-games/friday-night --player 1
```

2.4.2. 2. Dedicated VPS Host

Use this when one operator wants to run many games for many players on one server.

1. Run `scripts/install_vps.sh` as root.
2. Create each game under `/srv/ec/games/<slug>/` as the `ecgame` service user.
3. Register each game with `ec-sysop host games add` as root.
4. Run one `ec-sysop nostr serve` daemon for all games.
5. Run one `ec-sysop maint-all` timer for all games.

The standard VPS layout is:

```
/usr/local/bin/ec-game
/usr/local/bin/ec-sysop
/usr/local/bin/ec-gate-keys
/etc/ec-gate/config.kdl
/etc/ec-gate/identity.kdl
/var/lib/ec-gate/keys/
/srv/ec/games/<slug>/ecgame.db
```

The host-global relay URL and SSH address live in `/etc/ec-gate/config.kdl`. `scripts/install_vps.sh` writes them from `--relay`, `--ssh-host`, and `--ssh-port`. If you change those values later, edit `/etc/ec-gate/config.kdl` as root and restart `ec-nostr.service`.

If you self-host the relay on the same VPS, remember that the relay host must also be publicly reachable. A common setup is `nostr-rs-relay` bound to `127.0.0.1:8080` with Caddy or another HTTPS reverse proxy serving the relay hostname on port `443`.

2.4.3. 3. BBS Door Host

Use this when the sysop wants `ec-game` as a door under Mystic, ENiGMA½, or a similar BBS.

1. Create the game with `ec-sysop new-game`.
2. Reserve caller aliases with `ec-sysop settings reserve`.
3. Launch `ec-game` with a BBS dropfile.

4. Keep maintenance outside the door. Run `ec-sysop maint` from the host or the BBS event runner.

During the current beta, a BBS sysop should build from source or use a direct/private beta build. Windows BBS hosting is best-effort source-build only.

For the exact launcher setups, see:

- `docs/sysop/mystic-rust-setup.md`
- `docs/sysop/enigma-rust-setup.md`

2.5. Game Directory Layout

A hosted Rust game directory is DB-only:

```
/path/to/mygame/
  ecgame.db
```

All tools take `--dir /path/to/mygame` to locate the game.

2.6. Initializing a New Game

Create a new game with `ec-sysop new-game`:

```
sudo -u ecgame ec-sysop new-game /srv/ec/games/friday-night --name "Friday Night EC" --players 4
```

This creates one runtime file: `ecgame.db`.

> **IMPORTANT:** On a VPS host installed with `scripts/install_vps.sh`, create hosted games as the `ecgame` service user. If you create `/srv/ec/games/<slug>` as `root`, the `ec-nostr.service` daemon may fail to write session leases and hosted joins can time out.

The supported public creation flags are:

Flag	Type	Description
`--name`	string	Optional human-readable game title stored in `ecgame.db`. If omitted, `ec-sysop` derives a title from the directory slug.
`--players`	integer	Number of empires. Supported range: 1–25. Defaults to `4`.
`--year`	integer	Optional starting campaign year. Defaults to `3000`.
`--seed`	integer	Optional integer seed for the campaign RNG. Controls map layout, starting positions, and all random events. If omitted, the engine picks a random seed and saves it to `ecgame.db`. The seed cannot be changed after creation.

> **NOTE:** Use a different seed for every game. Reusing the same seed produces the same map, starting positions, and event sequence every time.

The target directory basename becomes the stable game slug. It must use only lowercase ASCII letters, digits, and dashes. The slug is distinct from the human-readable `game_name`, and both are distinct from the per-seat invite codes used by `ec-connect`.

2.7. Recommended Public Host

For new public campaigns, the recommended deployment path is `ec-sysop nostr` plus `ec-connect`. In that model, the sysop runs the normal Rust engine on a host, publishes the Nostr-facing daemon, and players join from their own machines with invite codes. The daemon handles identity and seat routing; the live game still runs inside `ec-game` over SSH. This keeps the original asynchronous EC rhythm without requiring per-player Unix account management or BBS door middleware.

If you are recruiting players for a live public game, point them first to esterianconquest.com. That landing page can carry the current public contact points and community links before you issue seat-specific invites. At the moment, it should direct them to the Discord channel at discord.gg/FMr8sfBa.

A minimal hosted setup looks like:

```
sudo -u ecgame ec-sysop new-game /srv/ec/games/friday-night --name "Friday Night EC" --
players 4
sudo ec-sysop host games add --config /etc/ec-gate/config.kdl --dir /srv/ec/games/
friday-night
sudo systemctl restart ec-nostr.service
ec-sysop nostr init
ec-sysop nostr serve
```

The values handed to players come from `/etc/ec-gate/config.kdl`:

- `relay` is the Nostr relay URL
- `ssh-host` and `ssh-port` are published in relay discovery during the first session handshake

For a self-hosted relay, that `relay` URL must already work from outside the box. If `nostr-rs-relay` is listening only on loopback, also run the public HTTPS reverse proxy for the relay hostname before expecting hosted joins to work.

After the daemon is running, view the hosted seat state and get the ready-to-distribute invite commands:

```
ec-sysop nostr seats --dir /path/to/mygame
```

The output lists every seat. Pending seats show the canonical join line:

```
Seat 1  [pending]
  ec-connect --join amber-river@relay.example.com

Seat 2  [claimed]
  npub1...
```

Send each player his `ec-connect --join ...` line. The player:

1. Runs `ec-connect`.
2. Presses `N`.

3. Pastes the invite code and presses Enter.

That is the full player-side flow. The invite carries the seat token and relay host, and `ec-connect` discovers the rest from the relay. No extra flags are normally required.

On first join, `ec-connect` creates or unlocks the player's encrypted identity and opens the SSH-backed `ec-game` session. The hosted seat is not claimed until the player actually saves the in-game empire name. If the player disconnects before that save, the invite is still pending and can be used again. After a completed first join, `ec-connect` caches the game locally, downloads the static starmap bundle, and returning players reconnect without re-entering any flags.

One hosted identity can claim only one seat in a given game. If the same wallet identity tries to redeem a second invite for that game, the daemon now rejects it and expects the player to reconnect with the already-claimed seat.

Power users and scripted workflows may also join directly from the command line:

```
ec-connect --join amber-river@relay.example.com
```

If an invite code is lost or compromised, reissue it:

```
ec-sysop nostr reissue --dir /path/to/mygame --player 2
```

This generates a fresh code for that seat, clears the old claim, and lets the player join again with the new code. On a normal host with the daemon config and identity present, `ec-sysop` now also republishes that game's public `30500` metadata immediately so the relay sees the new invite hash without waiting for a daemon restart.

If a player reports that a pending invite cannot be found on the relay, check and repair the published hosted metadata directly:

```
ec-sysop nostr verify --dir /path/to/mygame
ec-sysop nostr publish --dir /path/to/mygame
```

2.8. Hosted Player Identity Management

Hosted seats are bound to the first player identity that completes the in-game join and saves the empire name. In practice this means the seat is tied to one `npub` until the sysop changes it. Returning players should reconnect with the same local EC wallet identity they used when they finished that first join.

Players should not expect to paste the same invite into a brand-new wallet and take over an already-claimed seat. `ec-gate` treats that as a different player identity and rejects the join.

Players also should not expect one wallet identity to hold two seats in the same hosted game. Seat 1 and seat 2 in one campaign must be claimed by different identities, even if the same human is testing both paths.

If a player loses or forgets the original local identity, the supported recovery path is:

1. Reissue that seat with `ec-sysop nostr reissue`.
2. Send the player the new invite.
3. Have the player redeem it from the new wallet identity.

Reissuing is the deliberate “move this seat to a new identity” action. It clears the old `npub` binding and rotates the invite code at the same time.

Hosted seat claims are stored in `ecgame.db`. That SQLite state is the authority for invite codes, claim status, and bound player `npub`s. Legacy `roster.kdl` files are migration input only.

> **NOTE:** `/etc/ec-gate/config.kdl` is host-owned. Game-registry edits such as `host games add` and `host games remove` should be run as root. Restart `ec-nostr.service` after changing the game list so the daemon reloads the config.

3. Game Directory Structure

`ecgame.db` The SQLite runtime database. All game state lives here. Hosted seat claims live here too. Do not edit it by hand.

4. Configuration

Hosted Rust campaigns do not use a per-game `config.kdl`. Sysop-editable runtime policy now lives in SQLite alongside the rest of the campaign state. Use `ec-sysop settings ...` to inspect or change it:

```
ec-sysop settings show --dir /srv/ec/games/friday-night
ec-sysop settings set --dir /srv/ec/games/friday-night --game-name "Friday Night EC"
ec-sysop settings reserve --dir /srv/ec/games/friday-night --player 1 --alias SYSOP
```

4.0.1. Full Example

```
slug=friday-night
game_name=Friday Night EC
snoop=true
session_max_idle_minutes=10
session_minimum_time_minutes=0
session_local_timeout=false
session_remote_timeout=true
inactivity_purge_after_turns=0
inactivity_autopilot_after_turns=0
maintenance_enabled=true
maintenance_interval_minutes=10080
maintenance_next_due_unix_seconds=1775347200
reservation seat=1 alias=SYSOP
reservation seat=2 alias=NightShade
```

4.0.2. Stored Fields

Field	Type	Default	Description
`game_name`	string	`"Esterian Conquest"`	Display name shown in the main menu header.
`default_theme_key`	string	`"tokyo_night"`	Bundled color set used by default. This is a

			compiled-in key, not a file path.
`snoop`	bool	`#true`	Enable sysop snoop mode.
`reservations`	rows	*(absent)*	Optional BBS/dropfile seat reservations by caller alias.
`maintenance_enabled`	bool	`#true`	Whether `maint-all` should advance this game when it becomes due.
`maintenance_interval_minutes`	integer	`10080`	Maintenance cadence in minutes. `10080` = one week.
`maintenance_next_due_unix_seconds`	integer	*(auto)*	Next scheduled maintenance time as a Unix timestamp.

4.0.3. `session` Block

Field	Type	Default	Description
`max_idle_minutes`	integer	`10`	Minutes of inactivity before session timeout. Range: 0–120.
`minimum_time_minutes`	integer	`0`	Minimum session time guarantee in minutes. Range: 0–120.
`local_timeout`	bool	`#false`	Apply timeout to local (non-remote) sessions.
`remote_timeout`	bool	`#true`	Apply timeout to remote sessions.

4.0.4. `inactivity` Block

Field	Type	Default	Description
`purge_after_turns`	integer	`0`	Turns of inactivity before a player is purged. `0` = disabled. Range: 0–100.
`autopilot_after_turns`	integer	`0`	Turns of inactivity before autopilot engages. `0` = disabled. Range: 0–100.

4.0.5. `reservations` Block

Field	Type	Default	Description
`seat player=<N> alias="NAME"`	entry	*(absent)*	Reserve empire slot `N` for a BBS/dropfile caller alias. Alias matching is ASCII case-insensitive.

NOTE: `ec-sysop settings import-kdl --dir <game_dir>` still exists as a one-time migration tool for older beta campaigns that carried a per-game `config.kdl`. Normal hosted Rust campaigns do not depend on that file at runtime.

4.1. Display Defaults

The default display look is controlled by `default_theme_key`. That key names a compiled-in color set. It is not a file path.

Players may still choose a local color theme inside `ec-game`. That choice is stored in `ecgame.db` as a player preference.

5. SSH Access

The recommended hosted path above already uses SSH under the hood: players enter through `ec-connect`, and the daemon opens a PTY running `ec-game`. You can also run `ec-game` over SSH directly when you want a private shared-host setup, manual debugging, or a simple trusted deployment without the Nostr invite flow.

`ec-game` runs cleanly over SSH. No special flags are required for modern terminal sessions.

Color mode is auto-detected from the environment:

- `COLORTERM=truecolor` → 24-bit RGB
- `TERM` containing `256color` → 256-color
- Otherwise → 16-color ANSI fallback

Force a specific mode with `--color-mode` if your SSH setup does not propagate `COLORTERM` reliably:

```
ec-game --dir /path/to/mygame --player 1 --color-mode 256
```

UTF-8 encoding (the default) is correct for SSH sessions on modern terminals. Use `--encoding cp437` only if you are proxying through a BBS or a CP437 terminal emulator over SSH.

6. Local and Direct Play

Localhost play remains a fully supported secondary mode for solo campaigns, hotseat sessions, and sysop testing. Run `ec-game` directly in your terminal:

```
ec-game --dir /path/to/mygame --player 1
```

Color mode and encoding default to `auto` / `utf8`, which is correct for modern terminal emulators. The client detects `COLORTERM` and `TERM` automatically.

7. Legacy BBS Door Setup

`ec-game` can still run as a BBS door. This path is preserved for classic-host compatibility, not as the primary recommendation for new public campaigns. In door mode, the client reads from and writes to a socket instead of the local TTY, using CP437 encoding and 16-color ANSI.

7.1. Flags for Door Mode

```
ec-game \
  --dir /path/to/mygame \
  --player <1-based index> \
  --encoding cp437 \
  --color-mode ansi16
```

`--encoding cp437` switches box-drawing characters and other extended characters to the CP437 code page, which is required for correct rendering in classic ANSI-aware BBS terminals (SyncTERM, NetRunner, etc.).

When `--encoding cp437` is active, `--color-mode` defaults to `ansi16` automatically. Override only if you know your BBS clients support a richer color depth.

7.2. Drop File

`ec-game` can read a BBS drop file directly with `--dropfile <path>`:

```
ec-game \
  --dir /path/to/mygame \
  --dropfile /path/to/DOOR32.SYS
```

Supported drop file formats (auto-detected by filename, case-insensitive):

- `DOOR32.SYS` — modern standard (Enigma, Mystic, Talisman, etc.)
- `DOOR.SYS` — legacy, widest BBS software support
- `CHAIN.TXT` — WWIV format

The drop file supplies the player alias and session time limit. Explicit CLI flags always override drop file values. When `--dropfile` is given and `--encoding` is not, encoding defaults to `cp437` automatically.

`--timeout <minutes>` sets a session time limit independently of a drop file.

Reserve seats in `ecgame.db` when you want the caller alias to determine the empire automatically:

```
ec-sysop settings reserve --dir /path/to/mygame --player 1 --alias SYSOP
ec-sysop settings reserve --dir /path/to/mygame --player 2 --alias NightShade
```

With that in place, a reserved caller can launch with `--dropfile` alone. If the caller alias is not reserved, `--player` is still required. If both `--player` and `--dropfile` are supplied for a reserved caller, they must agree on the same empire slot.

> **NOTE:** The original DOS `ECGAME.EXE` v1.5 expects a strict 32-line WWIV-style `CHAIN.TXT` drop file. Enigma BBS-generated `DOOR.SYS` / `DORINFO` files will crash `ECGAME.EXE`. See `docs/sysop/enigma-bbs-setup.md` for the full legacy DOS door path if you need to host the original binary.

7.3. Enigma BBS (Rust Client)

The native Rust `ec-game` door is now verified on both Mystic and ENiGMA½. For ENiGMA, use the `abracadabra` module with `dropFileType: DOOR32`, `io: stdio`, and `encoding: cp437`. Pass

`--dir`, `--dropfile`, `--encoding cp437`, and `--color-mode ansi16` to the client, or use the helper wrapper at `tools/bbs/run_ec_rust.sh`. Use `--player` for unreserved callers, or reserve the alias in `ecgame.db` and let `--dropfile` resolve the seat.

If Enigma writes a `DOOR32.SYS`, you can pass it directly:

```
ec-game \
  --dir /path/to/mygame \
  --dropfile /path/to/DOOR32.SYS
```

For a fuller ENiGMA½ Rust-door setup, including a ready `abracadabra` configuration, see `docs/sysop/enigma-rust-setup.md`.

In BBS door mode, the reliable control contract is:

- `HJKL` for movement
- `^U` / `^D` for paging
- `Q` or `Esc` for back/quit

Do not rely on arrows or `PgUp` / `PgDn` for normal play through BBS hosts.

8. File-Based Turn Submission

For localhost, shared-host, or custom-client workflows, players can submit orders by writing a KDL turn file and passing it to `ec-game submit-turn`. Use `--check` to validate without writing, then run without it to apply:

```
ec-game submit-turn --check --dir /path/to/mygame --player 1 --file /path/to/player1-turn.kdl
ec-game submit-turn --dir /path/to/mygame --player 1 --file /path/to/player1-turn.kdl
```

The `--player` value must match the `turn player=...` header in the file. If any order in the file is invalid, the entire submission is rejected and nothing is written. This is a direct apply command, not a queue or upload inbox.

A minimal turn file:

```
turn player=1 year=3000
tax rate=37
```

A fuller example:

```
turn player=1 year=3000

tax rate=37

planet record=16 {
  build points=4 kind="scout"
}

fleet record=1 {
  order speed=3 kind="scout_system" x=16 y=13
}

message to=2 subject="Border" body="Watching the north lane."
```

For the full node reference and schema, see `docs/sysop/turn-kdl.md`.

9. Turn Processing and Maintenance

Run yearly maintenance with:

```
ec-sysop maint /path/to/mygame [turns]
ec-sysop maint-all [--config /etc/ec-gate/config.kdl]
```

`ec-sysop maint` advances the campaign in `ecgame.db`. EC does not schedule maintenance by itself. In a real deployment, invoke `ec-sysop maint` from your host scheduler or BBS tooling:

- a `systemd` timer
- `cron`
- a BBS event runner
- or manual sysop operation

For multi-game Nostr hosting, prefer a single global timer that runs `ec-sysop maint-all`. It reads the configured game directories from the gate config, skips games whose next due time has not arrived yet, and also skips games with live session leases so a player is never interrupted by maintenance.

Do not treat game settings as a scheduler. Snoop, inactivity, and the rest of the policy live in `ecgame.db`. The schedule still belongs to the host.

10. Player Management

Inactive-player policy is configured in `ecgame.db` under the campaign settings block. The two public thresholds are:

- `purge_after_turns`
- `autopilot_after_turns`

These values are runtime policy, not setup-time game creation input.

10.1. Reserving Seats

To reserve empire slots for specific BBS users:

```
ec-sysop settings reserve --dir /path/to/mygame --player 1 --alias SYSOP
ec-sysop settings reserve --dir /path/to/mygame --player 2 --alias NightShade
```

Each `seat` entry binds one 1-based empire slot to one caller alias. Alias matching is ASCII case-insensitive, so `NightShade`, `nightshade`, and `NIGHTSHADE` are treated as the same reservation.

With reservations in place, launch `ec-game` with `--dropfile` and let the caller alias choose the empire automatically:

```
ec-game --dir /path/to/mygame --dropfile /path/to/DOOR32.SYS
```

Important rules:

- if the caller alias is reserved, `--player` becomes optional
- if the caller alias is not reserved, `--player` is still required

- if both `--player` and `--dropfile` are supplied for a reserved caller, they must match
- if a reservation conflicts with an already-stored different player handle, `ec-game` will stop with a clear error so the sysop can reconcile the reservation and the runtime state

Reserving a seat does not by itself join or pre-name the empire. It only routes the caller to the intended slot. The usual first-time join flow still claims an open empire, and that first successful join records the caller alias into the player record for later logins.

11. Terminology

game directory The directory containing `ecgame.db` for one running game. Passed to all tools with `--dir`.

`ec-sysop` The public Rust command-line sysop tool for campaign creation maintenance, and Nostr hosting.

`ec-connect` The beta-quality player-side connection client for the recommended hosted flow. It manages the player's Nostr identity, joins games by invite code, downloads the static starmap bundle on first join, and opens the SSH-backed `ec-game` session.

`ec-game` The Rust TUI player client.

`ecgame.db` The SQLite database that is the runtime source of truth for the Rust engine.

campaign settings The sysop-managed runtime policy rows stored in `ecgame.db`. They control game name, snoop mode, the default compiled-in color set, seat reservations, maintenance cadence, and inactivity thresholds.

12. CLI Reference

12.1. ec-sysop

```
ec-sysop <subcommand> [options]
```

Subcommand	Purpose
`new-game`	Create a fresh DB-only campaign directory. Public flags: `--name`, `--players`, `--year`, and `--seed`.
`settings show|set|reserve|unreserve`	Inspect or edit the per-game runtime policy stored in `ecgame.db`.
`host games list|add|remove`	Inspect or edit the global game registry in `/etc/ec-gate/config.kdl`.
`host status`	Summarize the configured host, served game directories, claim counts, busy state, and maintenance-due state.
`nostr init`	Initialize the Nostr-hosting identity and config for the recommended public multiplayer path.
`nostr serve`	Run the Nostr-facing daemon that authenticates players and launches `ec-game` sessions.

`nostr seats`	List the hosted seat state stored in `ecgame.db` for one game directory.
`nostr reissue`	Generate a fresh invite code for one hosted seat, clear its old player binding, and republish that game's public `30500` metadata when possible.
`nostr publish`	Republish one game's public `30500` metadata to the configured relay immediately.
`nostr verify`	Compare one game's local hosted-seat state against the latest published `30500` on the configured relay.
`nostr migrate-roster`	Import a legacy `roster.kdl` into `ecgame.db`, copy its display name into the campaign settings rows, and archive the old roster file.
`maint`	Run one or more maintenance turns against `ecgame.db`.
`maint-all`	Sweep every game registered in the gate config, skipping games that are not due or that currently have active sessions.

12.2. ec-game

```
ec-game --dir <game_dir> [--player <1-based index>] [options]
ec-game submit-turn [--check] --dir <game_dir> --player <record> --file <turn.kdl>
```

Interactive client flags:

Flag	Description
`--dir <path>`	Game directory containing `ecgame.db`. Required.
`--player <N>`	1-based empire index. Required unless a reserved dropfile alias resolves the seat from `ecgame.db` campaign settings.
`--encoding <utf8|cp437>`	Output encoding. Default: `utf8`. Use `cp437` for BBS/door mode.
`--color-mode <ansi16|256|truecolor|auto>`	Color depth. Default: `auto` (env-detected). CP437 mode defaults to `ansi16`.
`--dropfile <path>`	Parse a BBS drop file (DOOR32.SYS, DOOR.SYS, or CHAIN.TXT). Supplies alias and timeout, defaults encoding to `cp437`, and can resolve the player seat through `ecgame.db` reservations. Explicit flags always override except that `--player` must match a reserved alias when both are present.

`--session-token <hex>`	Hosted-session lease token injected by `ec-gate` during Nostr/SSH login. Normal local and BBS launches do not pass this flag.
`--timeout <minutes>`	Session time limit in minutes. Overrides any drop file value.
`--queue-dir <path>`	Override turn queue directory. Default: `<game_dir>/queue`.

`submit-turn` flags:

Flag	Description
`--check`	Validate the KDL file without mutating the campaign.
`--dir <path>`	Game directory containing `ecgame.db`. Required.
`--player <N>`	1-based empire index. Required, and must match the KDL header.
`--file <path>`	Turn submission KDL file to validate or apply. Required.

NOTE: `ec-game submit-turn` is all-or-nothing. If any command in the file is invalid, the entire submission is rejected and nothing is written to `ecgame.db`.